Constraint-Flow Model (CFM) v1.0

A Diagnostic Layer for Structural Reasoning under Constraints

Natanaël van Dijk

March 30, 2026

Abstract

We formalize the Constraint-Flow Model (CFM), a diagnostic and interpretive framework for systems governed by a fail-closed, constitutionally grounded core (such as GSA). CFM models all phenomena as flows under constraints, where apparent structure is a stable pattern within a flow. Its role is strictly observational: it analyzes why a core yields a given outcome (admissible, halt, or refusal) and describes the relational dynamics of admissible flows. CFM is explicitly prohibited from altering core outputs, repairing failures, or introducing implicit assumptions. The relationship between CFM and the core is defined as a strict hierarchical separation: the core determines existence; CFM explains it. We formalize the core concepts of CFM—interpretative point, rupture, interference, and perspective distribution—and prove that CFM's analysis is truth-preserving with respect to the core's output. This establishes CFM as a safe, non-interfering diagnostic layer for structural reasoning systems.

1 Introduction

A reasoning system that only determines admissibility (like GSA) provides no insight into *why* a particular outcome occurred. In complex systems, understanding the relational dynamics and flow patterns that lead to a halt or refusal is essential for debugging, learning, and refinement. However, introducing such analysis must not compromise the system's core property: being fail-closed and free of implicit assumptions.

The Constraint-Flow Model (CFM) is designed to fill this gap. It provides a formal language for describing the behavior of a system—its flows, its constraints, and the points at which it ruptures—while remaining a purely *diagnostic* layer. CFM does not repair, guess, or override; it only observes, interprets, and explains.

This document defines CFM as a separate, non-interfering module that sits atop a fail-closed core. We establish the strict rules of engagement between the two and prove that CFM's operations cannot corrupt the core's deterministic, fail-closed semantics.

2 Core Principles of CFM

2.1 Flow as Primitive

Definition 2.1 (Flow)**.** *A* flow *is a finite or infinite sequence of states generated by a core transition operator* Θ_{core}*. Formally, given an initial state* S_0 *and a sequence of proposals* $q_1, q_2, \ldots$*, the flow is*

$$F_t = (S_0, S_1, \ldots, S_t)$$

where each $S_{i+1} = \Theta_{core}(S_i, q_{i+1})$ *if defined; otherwise the flow halts at the first undefined transition.*

Axiom 2.1 (Flow as Primitive)**.** *All phenomena are modeled as flows. What is perceived as static structure is a flow with a characteristic timescale that is slow relative to the observer. CFM's flows are projections of core transitions; they do not possess independent dynamics.*

2.2 Constraints as Selection Mechanism

Definition 2.2 (Constraint)**.** *A* constraint ω *is a predicate on a state that must evaluate to true for a flow to be admissible. A set of constraints* Ω_t *defines the admissible region at time* t*.*

Axiom 2.2 (Constraints Define Existence)**.** *A flow is only considered to exist (or be a valid structure) if it remains within the admissible region defined by its constraints at all times. This region is exactly the set of states for which the core returns* ADMISSIBLE*.*

2.3 Stability as Bounded Oscillation

Definition 2.3 (Stability)**.** *Let* $d : X \times X \to \mathbb{R}_{\geq 0}$ *be a metric on the configuration space* X*. A flow is* stable *towards an attractor* X^* *if for a given* $\epsilon > 0$*, the oscillation measure* $\sigma_t = d(S_t, X^*) \leq \epsilon$ *for all* t *within a time window.*

Axiom 2.3 (Structure as Stability)**.** *A structure is a flow that exhibits stability over a defined observational timescale.*

3 CFM as a Diagnostic Layer

3.1 Strict Separation of Roles

Axiom 3.1 (Hierarchical Non-Interference)**.** *The CFM layer is strictly subordinate to the core (e.g., GSA). CFM may read the core's state and output, but must never modify it or its decision process.*

Axiom 3.2 (No Correction or Repair)**.** *If the core outputs* HALT_SPEC_REQUIRED*,* REJECT*, or* REFUSAL*, CFM must not attempt to "fix" the input or infer missing information to make it admissible. Any such attempt would violate Axiom 4.*

3.2 CFM's Role: Describe, Explain, Interpret

Definition 3.1 (CFM's Purpose)**.** *The CFM layer analyzes a core's output to:*

1. ***Describe*** *the admissible and inadmissible flows.*
2. ***Explain*** *the reasons for a halt or refusal by identifying the violated constraints or undefined relations.*
3. ***Interpret*** *patterns of flows across multiple runs or time steps.*

3.3 Forbidden Actions

The following actions are *explicitly prohibited* for the CFM layer:

- **Override**: Declaring an inadmissible core output as admissible.
- **Repair**: Modifying a proposal or state to make it admissible.
- **Infer**: Adding implicit assumptions to fill missing specifications.
- **Decide**: Determining admissibility or the final system output.

4 Core CFM Constructs (for Analysis)

These constructs are used *only* by CFM to describe the system and are not part of the core's decision-making process.

4.1 Interpretative Point and Perspective

Definition 4.1 (Interpretative Point)**.** *An* interpretative point I^* *is a mapping from a state in the core's space S to an observed state in an observation space S_{obs}: $I^* : S \to S_{obs}$. It defines a specific "view" of the system.*

Definition 4.2 (Perspective Distribution)**.** *A* perspective distribution Π_t *is a probability distribution over possible interpretative points I^*, representing the CFM's epistemic uncertainty about which perspective is most explanatory for the observed flow. This distribution is not ontological; it reflects model selection for explanation.*

4.2 Rupture

Definition 4.3 (Rupture)**.** *A* rupture *occurs at time t if the core's transition would leave the admissible region defined by Ω_t. Formally,*

$$\textit{Rupture} \iff \textit{core_output} \in \{\textsc{Reject}, \textsc{Refusal}\}.$$

(Note: Halt_Spec_Required *indicates missing specification, not a rupture from an existing admissible region.)*

4.3 Rupture Direction and Interference

Definition 4.4 (Rupture Direction)**.** *Let the last admissible state be S_t and the attempted next state be S_{t+1} (which caused the rupture). The* rupture direction *is the normalized displacement in configuration space:*

$$v_r = \frac{S_{t+1} - S_t}{\|S_{t+1} - S_t\|},$$

if the configuration space is metric; otherwise it is defined as the qualitative direction of change in the relevant variables.

Definition 4.5 (Interference)**.** *The* interference *of a constraint ω with the rupture direction v_r measures the sensitivity of ω to a small step in direction v_r. For a differentiable ω, this is $|\nabla\omega(S_t) \cdot v_r|$. In discrete or symbolic settings, it is defined as the change in ω's truth value (or a measure of violation) when moving a minimal distance along v_r. The constraint with highest interference is the primary driver of the rupture.*

4.4 Learning the Perspective Distribution

Definition 4.6 (Bayesian Update)**.** *CFM updates its perspective distribution using Bayes' rule based on observed evidence e_t (e.g., the core's output):*

$$\Pi_{t+1}(I^*) \propto P(e_t \mid I^*) \cdot \Pi_t(I^*),$$

where $P(e_t \mid I^*)$ encodes how well perspective I^* explains the evidence.

5 Relationship to the Core (GSA)

Theorem 5.1 (CFM as a Restriction)**.** *The core (GSA) is a special case of CFM where the perspective distribution Π_t is a point mass on a fixed, constitutional interpretative point, no rupture transitions occur (constraints are static), and no learning takes place.*

Proof. By construction, GSA's symbolic state space S, invariant hierarchy Φ_i, and transition operator Θ define a static constraint set $\Omega = \Phi_0 \cup \cdots \cup \Phi_k$. GSA's single "constitutional" perspective corresponds to a fixed I^*, and its fail-closed pipeline (RGR, RCR, kernel) is a static admissible region. This aligns with Axiom 4 of CFM, which prohibits CFM from interfering with such a static, deterministic core. □

Theorem 5.2 (Diagnostic Preservation)**.** *For any core output O, the CFM layer's analysis, based on core-observable data, will not change O. CFM's operations are truth-preserving with respect to the core's deterministic semantics.*

Proof. Let $O = \text{Core}(S, q)$ be the core's deterministic output. CFM's analysis is a function f of the core's state and output: $\text{CFM} = f(S, O)$. Since f does not modify S or q, and the core does not depend on f, the output O is independent of CFM. Hence $\partial O / \partial \text{CFM} = 0$. □

6 Formal Rules of Engagement

The following rules are enforced to maintain the strict separation:

1. **Read-Only Access**: CFM may read the current core state and output log, but may not write to any core state variables.
2. **No Output Substitution**: The system's final output is determined solely by the core. CFM's analysis may be appended as metadata but cannot replace or modify it.
3. **No Generative Repair**: If the core halts with Halt_Spec_Required, CFM must not generate the missing specification and re-run the core. Any such action constitutes a new, separate system run.
4. **Diagnostic Purpose Only**: The results of CFM analysis (e.g., rupture direction, interference, perspective distribution) are for external interpretation only and are not used as input to the core's decision logic.

7 Conclusion

CFM is a formal diagnostic framework that sits above a fail-closed reasoning core. It provides the language and tools to analyze system behavior—flows, ruptures, and perspectives—without compromising the core's integrity. The strict separation of roles (core decides; CFM describes) ensures that the system remains fail-closed, deterministic, and free from the very implicit assumptions that CFM is designed to identify. This architecture allows for a system that is both reliable (due to its core) and interpretable (due to its diagnostic layer).

Acknowledgments

This work was developed through iterative refinement supported by multiple large language models, including systems such as ChatGPT, Claude, and Gemini. These systems were used as tools for formalization, critique, and structural validation. All core concepts, definitions, and architectural principles originate from the author. The role of these systems was strictly assistive: to test consistency, expose implicit assumptions, and accelerate the formalization process.

A Simulation Plan for CFM Prototype

A minimal 1D simulation is planned to test the formal definitions:

- **Core**: A simple fail-closed core with state $x \in \mathbb{R}$ and constraints $\omega_1 : x \geq -1$, $\omega_2 : x \leq 1$. Transitions are proposals to move by Δx; the core returns Admissible if the new state satisfies both constraints, Reject otherwise.
- **Flow**: Sequence of core transitions recorded by CFM.
- **Rupture detection**: When core returns Reject.

- **Interference**: Compute for each constraint the absolute change in its violation measure (e.g., distance to boundary) per unit step in the proposed direction. The constraint with largest interference is the primary rupture cause.
- **Perspective distribution**: Two perspectives: "normal" (observes x directly) and "inverted" (observes $-x$). Update Π using Bayes after each transition based on whether the core's output matches the perspective's prediction.

This prototype will demonstrate that CFM can analyze core behavior without interfering, and that the formal language (flow, rupture, interference, perspective) is operationally realizable.